1. 과제 개요

1.1 연구 배경 및 필요성

CNN은 영상 인식에 널리 사용되지만, 추론 과정에서 반복되는 convolution과 activation·weight의 이동은 많은 에너지를 요구한다. 전력과 발열이 제한된 edge 환경에서는 요구 처리 속도를 만족하면서 추론당 에너지를 줄이는 것이 중요하다. 이를 위해서는 연산기를 병렬화하여 처리량을 높이는 것과 함께, 수행할 연산의 수와 데이터 이동 자체를 줄여야 한다. Eyeriss는 연산보다 데이터 이동에 더 많은 에너지가 소모될 수 있음을 보이고, 데이터 재사용과 gating을 활용한 저전력 처리의 필요성을 제시하였다 [1].

3×3 convolution은 이러한 연산 최적화의 중요한 대상이다. VGG는 작은 3×3 kernel을 반복하여 깊은 네트워크를 구성하며 [7], ResNet의 bottleneck block도 1×1 convolution으로 channel 수를 조절한 뒤 3×3 convolution으로 공간 정보를 처리한다 [8]. 따라서 작은 kernel이 여러 layer와 channel에 걸쳐 반복될 때, convolution당 필요한 multiplication을 줄이는 것은 CNN 추론의 연산 부담을 낮추는 방법이 된다. Winograd는 인접 출력의 계산을 tile 단위로 묶어 이러한 multiplication 수를 줄인다 [2].

Activation sparsity도 활용할 수 있다. CNN에서 널리 사용하는 ReLU는 음수 값을 0으로 만들므로, 다음 layer에는 zero activation이 포함된 feature map이 전달된다. 이때 0과의 multiplication은 출력에 기여하지 않는다. SCNN은 이러한 activation sparsity를 활용하여 불필요한 연산과 데이터 전달을 줄이고 성능 및 에너지 효율을 개선하였다 [9]. 즉, zero activation을 검출하고 유효한 데이터의 연산을 모아 처리하면 실제 실행해야 하는 작업을 더 줄일 수 있다.

이에 본 과제는 3×3 convolution의 multiplication을 줄이는 Winograd 연산과, 변환 결과의 zero row·tile을 건너뛰는 **Sparsity-Aware Winograd CNN Accelerator** 를 제안한다. 변환과 후속 처리 사이의 중간 저장을 줄이고, 사용하지 않는 register의 클럭 활동도 억제하여 연산 감소가 실제 회로의 에너지 절감으로 이어지도록 가속기 아키텍처를 설계하였다.

1.2 제안 아키텍처와 기여

제안 아키텍처는 tile 생성, 2-stage transform, 공유 16-lane MAC 및 fused output 처리를 하나의 데이터 흐름으로 구성한다. V buffer는 zero row를 제외한 channel별 데이터를 압축하여 MAC에 공급하고, 전체 zero 작업은 우회한다. Tile 단위 후속 연산과 activation lifetime에 따른 저장 공간 재사용으로 데이터 이동 및 저장 비용을 줄이며, ICG로 데이터 갱신이 없는 register의 클럭 활동을 억제한다.

2. 설계 목표

설계 목표는 요구 처리 속도와 정수 연산 정확도를 만족하면서, convolution의 multiplication, zero 데이터의 불필요한 MAC 연산, 중간 데이터 저장 및 클럭 활동을 줄이는 것이다. 이를 위해 각 연산 단계의 데이터 흐름과 register 경계를 설계하고, 고정 속도 입력 수락, 실행 사이클, 1 GHz timing 및 활동 기반 전력·에너지로 구현 결과를 평가하였다.

2.1 Winograd Convolution 원리

일반적인 direct convolution은 kernel을 한 칸씩 이동시키며 각 출력 위치를 독립적으로 계산한다. 이에 비해 Winograd convolution은 서로 인접한 여러 출력을 하나의 tile로 묶어 계산함으로써 convolution에 필요한 multiplication 수를 줄이는 알고리즘이다 [2].

Fig. 1에서 중첩되는 4×4 input tile과 서로 겹치지 않는 2×2 output tile의 대응 관계를 볼 수 있다.

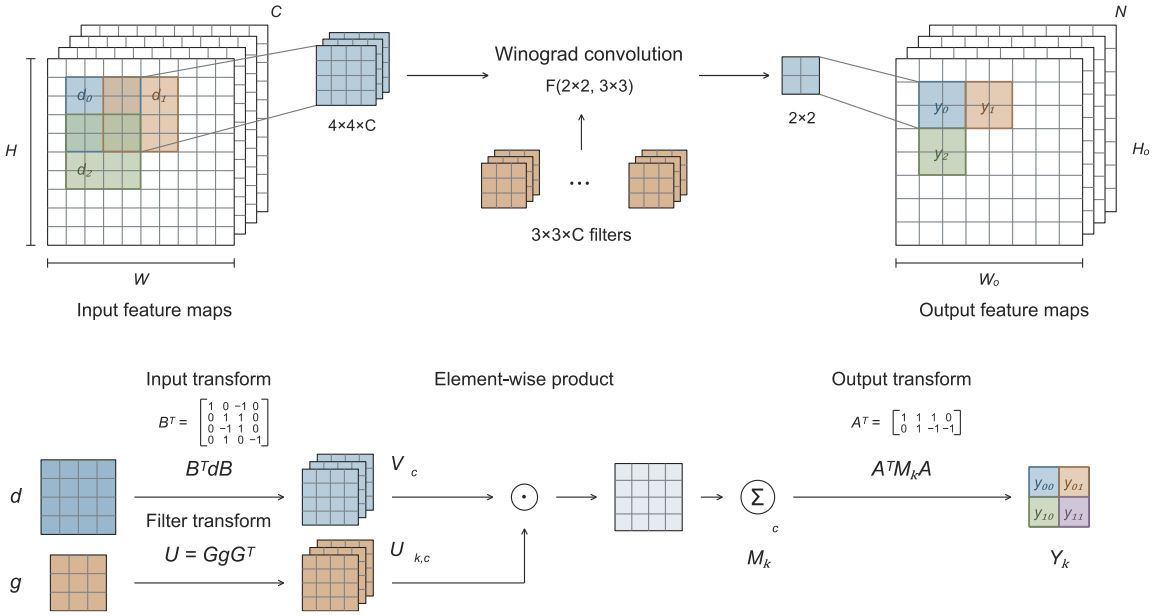


Fig. 1. Winograd F(2×2, 3×3) tile mapping and transform-domain computation.

Winograd 알고리즘은 일반적으로 $F(m \times m,\ r \times r)$로 표기한다. 여기서 $\mathrm{r} \times \mathrm{r}$은 convolution kernel의 크기이며, $\mathrm{m} \times \mathrm{m}$은 한 번의 연산으로 생성되는 output tile의 크기이다. 연산에 필요한 input tile의 한 변 크기 $\boldsymbol{\alpha}$는 다음과 같다.

$$\alpha = m + r - 1 \tag{1}$$

본 설계에서 사용하는 $F(2 \times 2, 3 \times 3)$에서는 $m = 2,\ r = 3$이므로 4×4 input tile과 3×3 weight로부터 2×2 output tile을 생성한다.

Winograd convolution의 연산 과정은 다음과 같이 표현된다.

$$U = GgG^T \tag{2}$$

$$V = B^T dB \tag{3}$$

$$M_k = \sum_c U_{k,c} \odot V_c \tag{4}$$

$$Y_k = A^T M_k A \tag{5}$$

각 기호의 의미는 다음과 같다.

- d: 4×4 input activation tile
- g:3×3 spatial weight
- V: transform-domain activation
- U: transform-domain weight
- M: element-wise multiplication과 input-channel reduction 결과
- Y: 복원된 2×2 output tile
- A, B, G: 고정된 Winograd transform matrix
- $\odot$: element-wise multiplication

먼저 input activation과 weight를 각각 Winograd domain으로 변환한다. 변환 결과인 V와 U는 모두4×4크기이며, 같은 위치의 원소끼리 총 16회의 multiplication을 수행한다. 여러 input channel이 존재하면 channel별 $U \odot V$결과를 동일한 위치끼리 더하여 channel reduction을 수행한다. 마지막으로 output transform을 적용하여 2×2결과 Y를 복원한다.

Direct 3×3 convolution으로 2×2 출력을 계산하려면 input/output channel 조합마다 다음과 같이 36회의 multiplication이 필요하다.

반면 Winograd $F(2 \times 2,3 \times 3)$에서는 4×4 transform domain의 element-wise multiplication만 필요하므로 이를 16회로 줄일 수 있다.

Weight transform은 inference 전에 수행할 수 있으므로, 본 가속기는 spatial weight g가 아니라 미리 변환 및 양자화된 U를 model parameter로 입력받는다. Bias, ReLU, pooling 및 requantization은 Winograd 수식에 포함되지 않으며, Y가 복원된 이후 Fused Output Unit에서 처리한다.

2.2 검증용 CNN과 구현 범위

MNIST는 제안 아키텍처의 정수 연산과 전체 추론 동작을 검증하는 대상으로 사용하였다. 본 절에서는 대회 interface와 비교 기준에 맞춰 구현한 CNN의 구성을 명시한다.

Baseline CNN은 두 개의 5×5 convolution layer와 두 개의 max-pooling layer, 하나의 FC layer로 구성된다. 본 설계에서는 전체 layer 순서와 channel 수는 유지하면서 convolution kernel을 5×5에서 3×3으로 변경하였다.

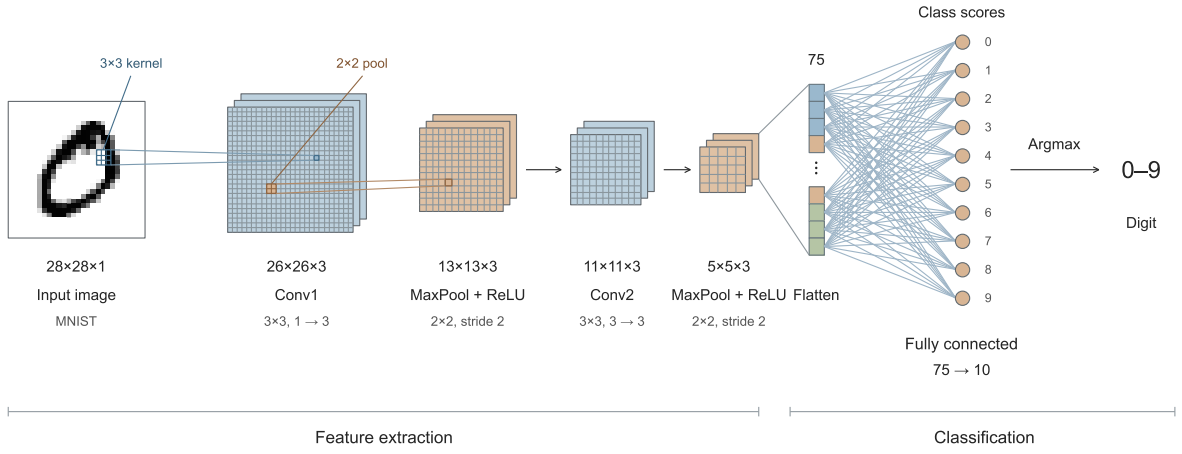


Fig. 2. Target CNN for MNIST classification.

Layer	Baseline (5×5)	Proposed (3×3)
Input	1×28×28	1×28×28
Conv1 output	3×24×24	3×26×26
Pool1 output	3×12×12	3×13×13
Conv2 output	3×8×8	3×11×11
Pool2 output	3×4×4	3×5×5
FC input	48	75
FC output	10	10

Table 1. Comparison of the baseline and proposed CNN architectures.

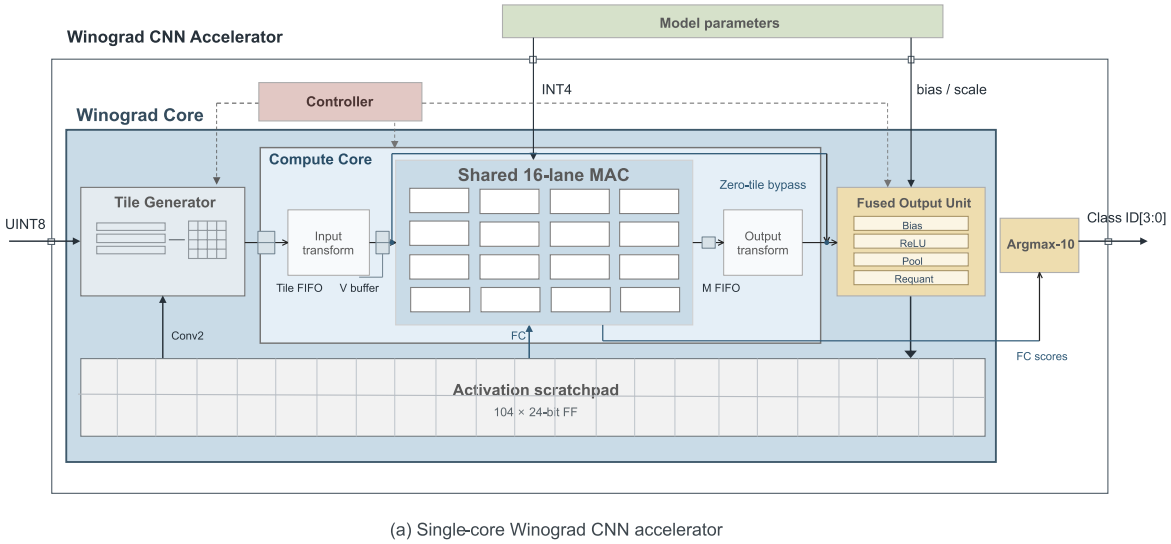
5×5 kernel에도 Winograd 알고리즘을 적용할 수 있다. 그러나 5×5 kernel로 2×2 output tile을 생성하는 $F(2 \times 2,\ 5 \times 5)$ 에서는 식 (1)에 따라 input tile의 크기가 6×6으로 증가한다. 따라서 transform-domain element 수는 $F(2 \times 2,\ 3 \times 3)$의 16개에서 36개로 증가한다. 이는 multiplier 수뿐만 아니라 transform 연산기, 중간 register, buffer 및 datapath bit-width 증가로 이어진다.

Kernel 변경에 따라 convolution과 pooling 이후의 activation 크기도 달라진다. 마지막 pooling 결과는 3×4×4에서 3×5×5로 변경되며, FC 입력은 48개에서 75개로 증가한다. 이러한 모델 변경이 정확도에 미치는 영향은 PyTorch FP32 사전학습을 통해 먼저 확인하고, 이후 Winograd-aware QAT와 정수 연산 검증을 수행하였다.

본 RTL은 Table 1의 channel 수, 주소 및 layer 순서에 맞춰 구현하였다. 다른 CNN에 적용할 때에는 해당 차원에 맞는 저장 용량과 제어 구성을 정해야 하며, 3×3 convolution의 타일 연산 원리와 compute dataflow를 바탕으로 확장할 수 있다.

3. 설계 및 시뮬레이션

3.1 Hardware Architecture 및 Dataflow



(a) Single-core Winograd CNN accelerator

Fig. 3. Proposed Winograd CNN accelerator with Zero-tile bypass.

Fig. 3의 구조는 4×4 input tile을 받아 2×2 output tile을 생성하는 Winograd Core를 중심으로 한다. 16개 transform-domain 위치를 병렬 처리하는 MAC을 공유하고, layer와 input channel은 시간적으로 재사용한다. Controller가 실행 단계를 관리하며 weight, bias 및 requantization parameter는 외부 interface에서 공급한다.

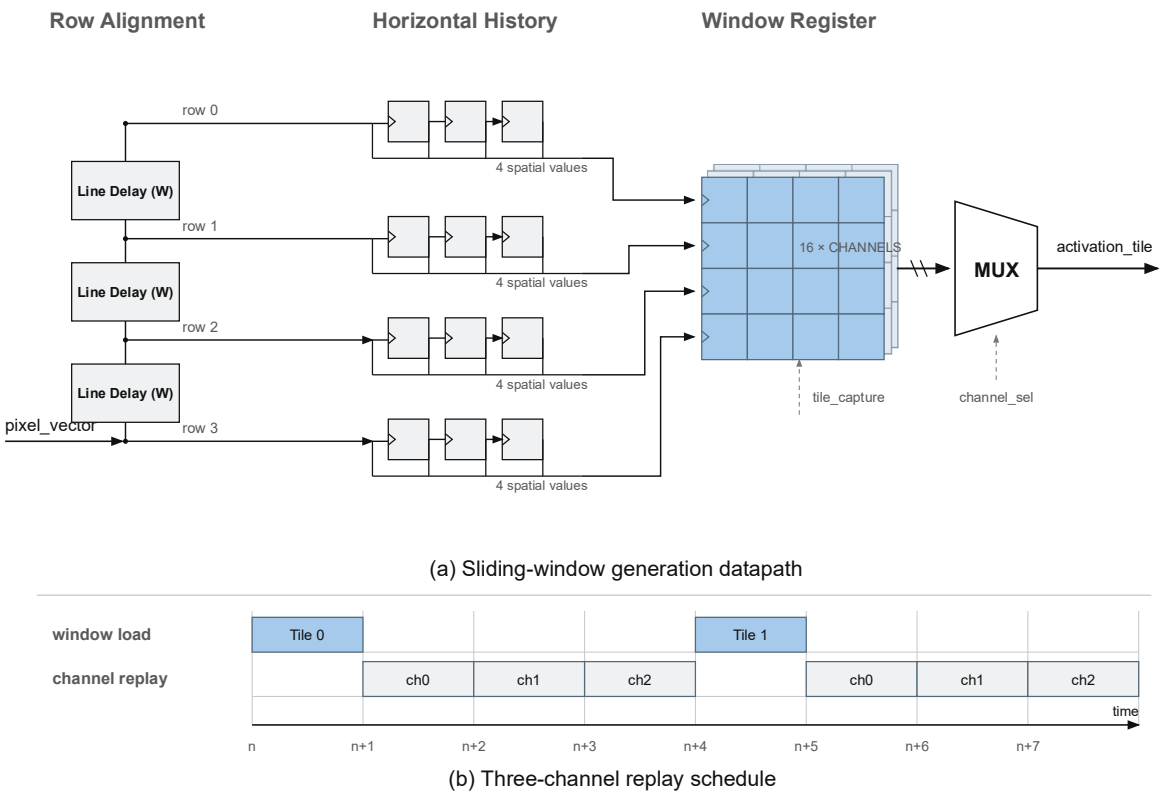
Conv1은 외부 pixel stream을 사용하고, Conv2는 Activation Scratchpad의 Conv1 결과를 다시 읽는다. Conv2가 끝나면 같은 MAC을 FC mode로 전환하므로 layer별 연산기를 별도로 두지 않는다.

3.2 Winograd Core Microarchitecture

3.2.1 Tile Generator

Tile Generator는 영상을 왼쪽에서 오른쪽으로, 위에서 아래로 한 pixel씩 받아 line buffer와 가로 방향 이력 register에 저장한다. 인접 window에서 겹치는 activation을 재사용하여 4×4 tile을 생성한다.

Conv1에서는 외부 pixel stream이 직접 입력되고, Conv2에서는 Activation Scratchpad에서 읽은 세 channel의 packed activation이 입력된다. Channel selector는 실행 중인 input channel에 해당하는 byte를 선택하며, line buffer와 shift register에 필요한 activation이 채워진 시점에 하나의 4×4 tile을 출력한다.



(a) Sliding-window generation datapath

(b) Three-channel replay schedule

Fig. 4. Sliding-window Tile Generator and channel replay.

3.2.2 Input Transform

Input Transform은 4×4 spatial tile d에 $B^T dB$ 연산을 적용하여 transform-domain activation V를 생성한다. 본 설계에서 사용하는 F(2×2, 3×3)의 input transform matrix B^T는 다음과 같다 [2].

$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix} \tag{6}$$

Matrix의 모든 coefficient가 0과 ±1이므로 constant multiplication은 wire selection, addition 및 subtraction으로 치환할 수 있다. 하나의 4-element vector $[d_0, d_1, d_2, d_3]$에 대한 1-D transform은 다음과 같다.

$$t_0 = d_0 - d_2, \qquad t_1 = d_1 + d_2, \qquad t_2 = d_2 - d_1, \qquad t_3 = d_1 - d_3$$

따라서 Input Transform에는 일반적인 multiplier가 필요하지 않으며 네 개의 add/subtract datapath로 구현할 수 있다. 2-D transform은 column 방향의 1-D transform을 수행하여 intermediate result를 TEMP Register

Bank에 저장한 후, 동일한 연산을 row 방향으로 적용하는 2-stage pipeline으로 구성하였다. 두 transform stage 사이에는 register boundary를 배치하여 combinational path를 분리하였다.

Winograd를 적용한 기본 구조는 zero row가 없는 tile도 16개 lane으로 처리한다. 여기에 Fig. 5와 같이 변환 결과의 zero row를 이용하면 활성 lane 슬롯을 더 줄일 수 있다. 두 행이 0이면 8개 lane만 사용하고, 전체 channel group이 0이면 작업을 우회한다.

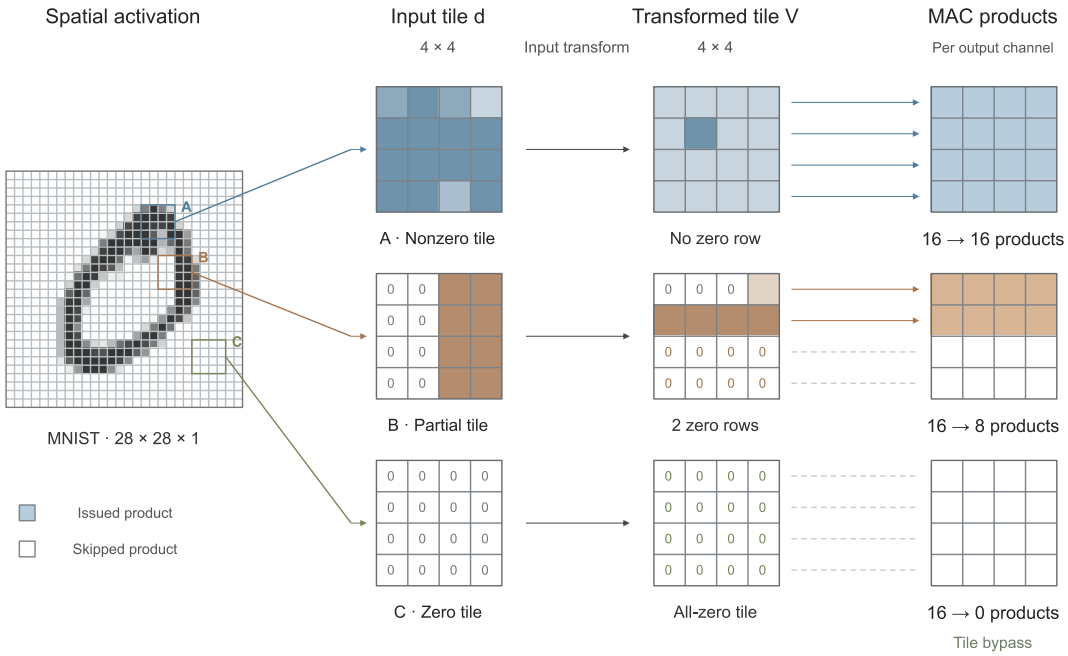


Fig. 5. Activation examples and row-wise MAC work reduction for one input/output-channel pair.

입력 변환의 중간값을 T = BᵀD, 최종 변환 결과를 V = TB라고 하자. B는 가역이므로 T의 한 행이 영벡터인 조건과 V의 같은 행이 영벡터인 조건은 동치이다. 따라서 TEMP 단계에서 zero row를 판정하고, 그 결과를 V 및 channel 정보와 함께 전달할 수 있다. 본 입력 범위에서는 INT10 TEMP와 INT11 V에 overflow가 발생하지 않으므로 중간값을 이용한 판정은 실제 V의 zero-row 판정과 일치한다.

$$T = B^{\mathrm{T}}d, \quad V = TB, \quad T[r,:] = 0 \iff V[r,:] = 0$$

Fig. 6과 같이 각 TEMP 행의 네 INT10 원소, 즉 40-bit 전체를 reduction NOR로 검사하여 zero-row mask를 생성하였다. 별도의 transform 없이 기존 중간값을 이용하며, mask와 V를 같은 valid/ready 조건에서 유지하여 stall 중에도 대응 관계를 보존한다.

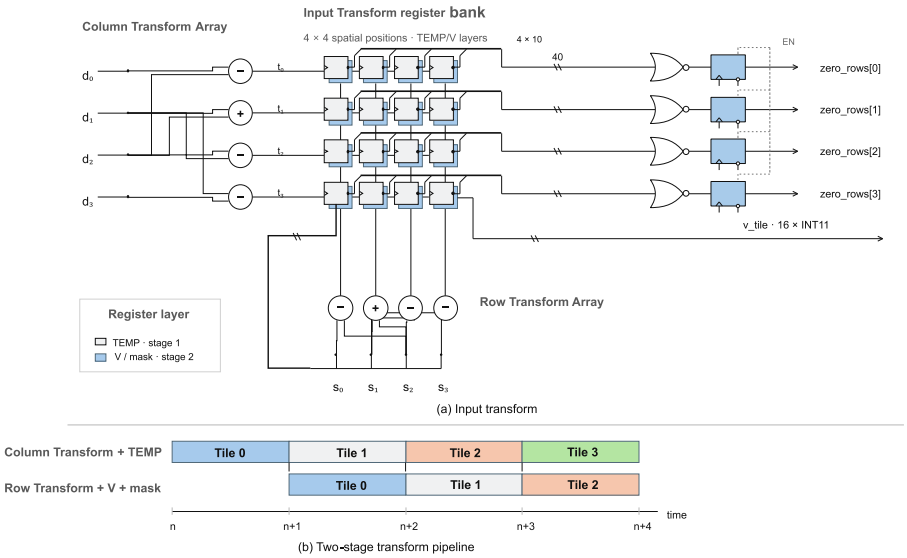


Fig. 6. Zero-row detection using TEMP registers and reduction NOR.

3.2.3 V buffer 및 Zero-tile bypass

각 output channel은 같은 공간 tile에 속한 input channel들의 기여를 더해야 한다. V buffer는 행마다 독립적인 unread channel 집합을 유지하고, zero row를 제외한 첫 channel을 선택한다. 따라서 한 번의 MAC 입력에서 첫 번째 행은 channel 0, 두 번째 행은 channel 1을 선택하는 식으로 서로 다른 channel의 행을 함께 배치할 수 있다. 각 행에는 선택한 channel과 같은 U 행을 공급하여 수학적인 channel reduction을 유지한다.

Fig. 7(a)의 예에서는 세 channel의 유효 행을 두 번의 MAC 입력으로 묶는다. 첫 입력은 16개 lane을 사용하고, 두 번째 입력에서는 첫 행의 4개 lane만 활성화하며 나머지 accumulator는 값을 유지한다.

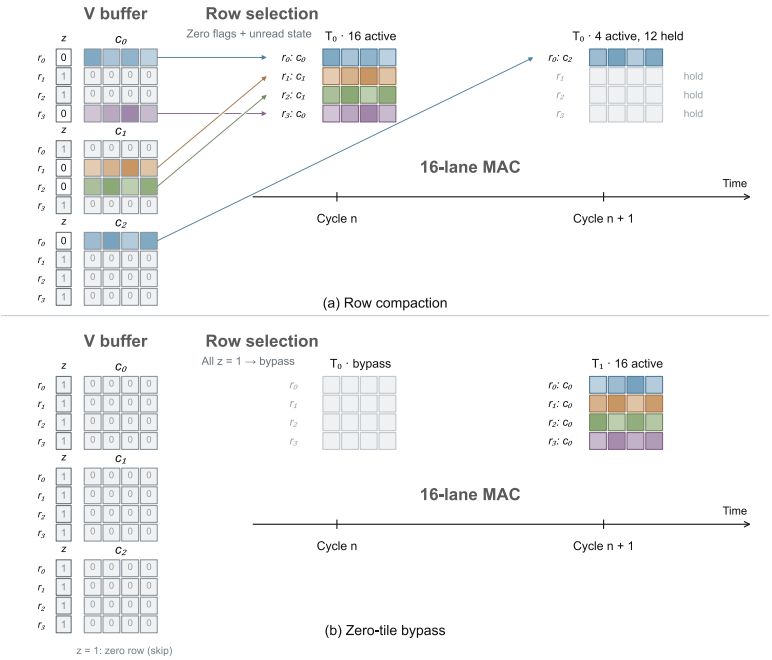


Fig. 7. Channel-row compaction and ordered whole-zero job bypass.

전체 channel group의 네 행이 모두 0이면 MAC과 Output Transform을 수행할 필요가 없다. 이 경우 Fig. 7(b)와 같이 zero-job 정보를 ordering 경로로 전달하고 다음 tile을 정상 처리한다. 다만 bias, ReLU 및 requantization은 여전히 필요하므로 최종 activation을 무조건 0으로 쓰지는 않는다. 일반 작업과 우회 작업의 순서를 보존하여 bias와 output-channel 정보가 다른 결과에 적용되지 않도록 하였다.

Algorithm 1	**Row compaction and whole-zero job bypass**
Input	Channel tiles V[c], zero-row flags z[c,r], weights U[k,c]
Output	Ordered reductions M[k] or zero-job metadata
1	for each output channel k do
2	R[r] ← channels c with z[c,r] = 0, for r = 0…3
3	if all R[r] are empty then
4	enqueue zero-job(k); continue
5	first ← true
6	while at least one R[r] is nonempty do
7	select the first unread c[r] in each nonempty R[r]
8	last ← no unread channel remains after these selections
9	issue matching V[c[r]][r,:] and U[k,c[r]][r,:] rows
10	hold selections, data, first and last until accepted
11	remove accepted c[r] from R[r]; first ← false
12	end while
13	end for

Algorithm 1. Row compaction with ordered whole-zero bypass.

3.2.4 Reconfigurable 16-lane MAC

16-lane MAC은 Winograd convolution과 FC layer에서 공유된다. Convolution mode에서 각 lane은 4×4 Winograd domain의 한 position을 담당한다. 각 lane은 INT11 activation V와 INT4 weight U를 곱하고, input channel에 따라 결과를 INT18 accumulator에 누적한다.

Conv1은 input channel이 하나이므로 별도의 channel accumulation 없이 결과를 생성한다. Conv2에서는 각 행의 유효 input channel을 순서대로 선택하고, 같은 Winograd 위치의 결과를 INT18 accumulator에 누적한다. 공통 first/last 정보로 한 output-channel 작업의 초기화와 종료를 표시하며, 마지막 유효 행까지 처리한 뒤 RESULT register를 갱신한다.

FC mode에서는 동일한 UINT8 activation을 lane 0–9에 공급하고 각 class에 대응하는 INT4 weight를 곱한다. 각 lane은 75개 feature의 결과를 누적하며, 마지막 cycle에 INT16으로 저장한 FC bias를 sign-extension하여 더한다. 사용하지 않는 lane 10–15는 비활성화된다.

Weight가 0이거나 lane이 비활성 상태인 경우에는 multiplier 입력을 0으로 고정하는 operand isolation을 적용하였다. 연산 cycle과 pipeline timing은 유지하면서 불필요한 data switching이 multiplier 내부로 전파되는 것을 억제하였다.

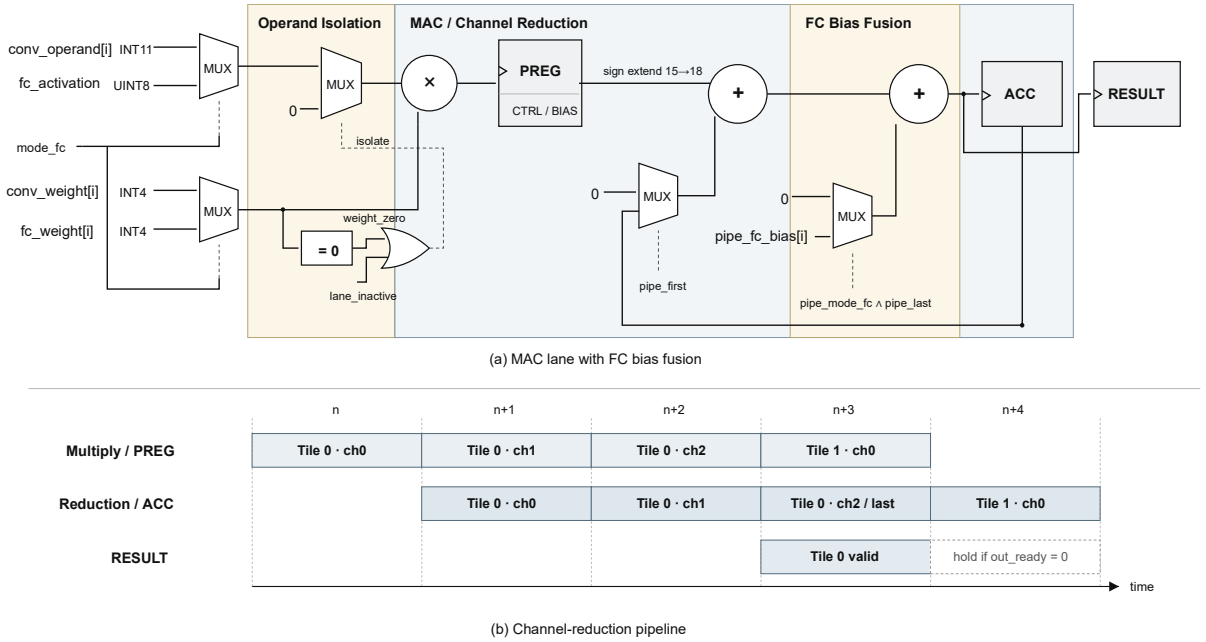


Fig. 8. Reconfigurable MAC lane with FC bias fusion and pipeline schedule.

3.2.5 Output Transform

Output Transform은 channel reduction이 완료된 4×4 intermediate result M 에 $A^{\mathrm{T}}MA$ 연산을 적용하여 2×2 spatial output tile Y를 생성한다. 사용된 output transform matrix A^{T}는 다음과 같다 [2].

$$A^{T} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix} \tag{7}$$

A^{T} 역시 coefficient가 0과 ± 1로만 구성된다. 하나의 4-element vector $[m_0, m_1, m_2, m_3]$에 대한 1-D output transform은 다음과 같다.

$$l_0 = m_0 + m_1 + m_2, \qquad l_1 = m_1 - m_2 - m_3$$

따라서 Output Transform도 multiplier 없이 add/subtract unit만으로 구현할 수 있다. 첫 번째 stage에서는 $A^{\mathrm{T}}M$을 계산하여 2×4 intermediate result를 Register Bank에 저장하고, 두 번째 stage에서는 오른쪽 matrix A를 적용하여 최종 2×2 output tile을 생성한다. 각 addition과 subtraction에는 INT18 saturation을 적용하여 hardware와 integer inference model의 numerical behavior를 일치시켰다.

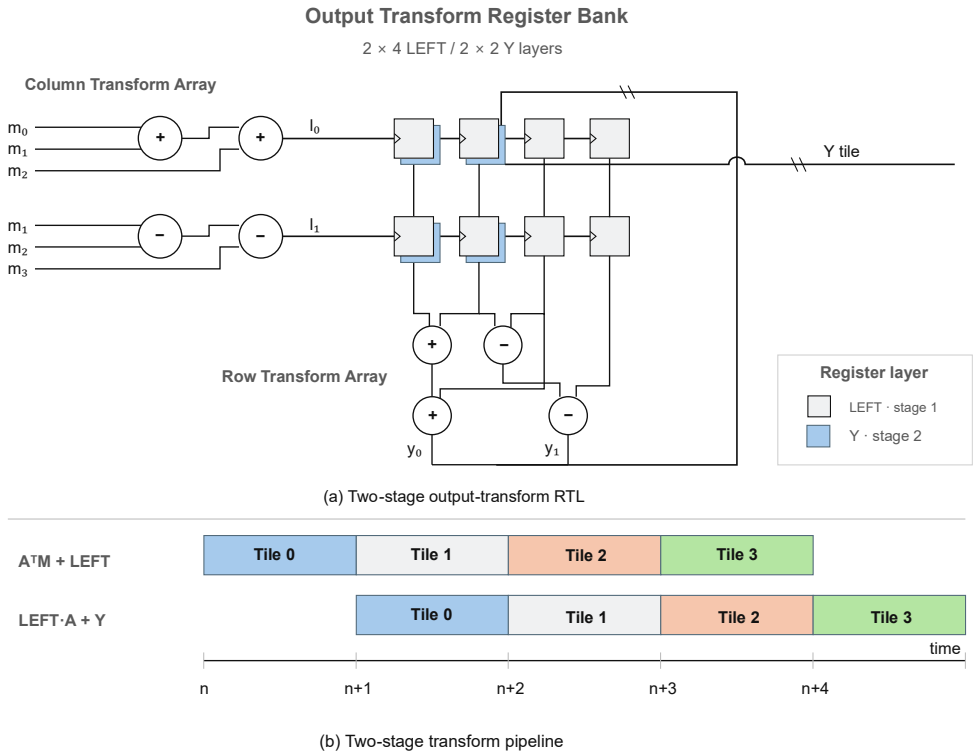


Fig. 9. Two-stage Output Transform microarchitecture.

3.2.6 Fused Output Unit

Multiplication을 줄인 뒤에도 full-precision feature map을 저장하고 다시 읽는다면 intermediate memory의 비용은 남는다. Fig. 10의 Fused Output Unit은 Output Transform의 2×2 결과를 bias, ReLU, pooling 및 requantization으로 직접 전달하고 다음 layer의 UINT8 activation만 저장하도록 구성하였다 [3].

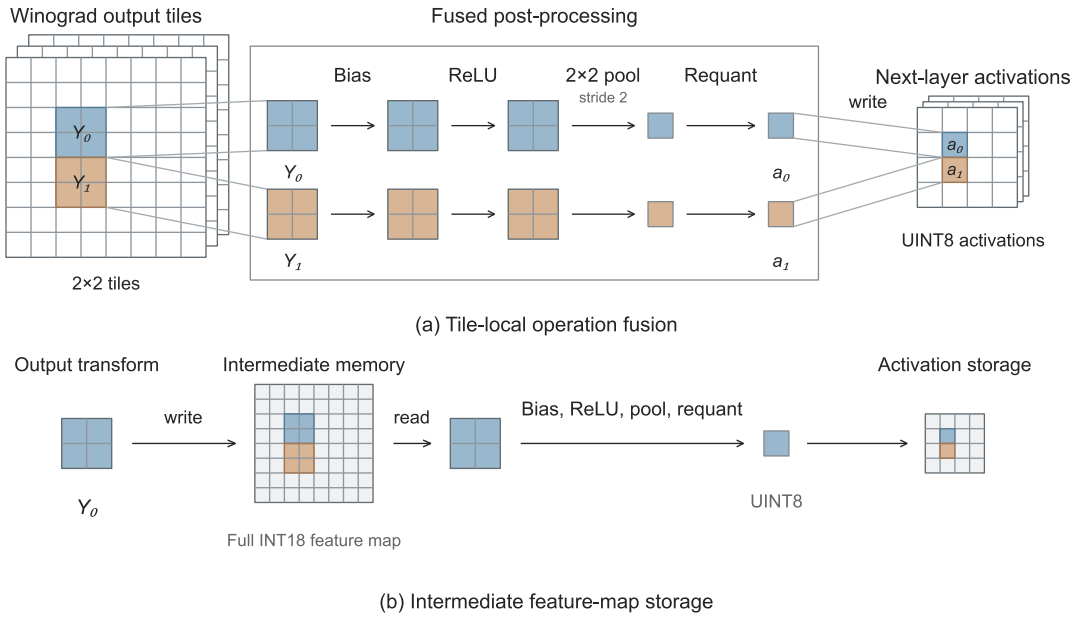


Fig. 10. Conceptual tile-local fusion and removal of intermediate feature-map storage.

Requantization은 accumulator-domain result에 layer별 integer multiplier를 곱한 후 고정된 shift를 적용한다. 음수는 ReLU에 의해 0으로 제한하고, UINT8 범위를 초과하는 값은 255로 saturation한다.

RTL에서는 네 값에 같은 output-channel bias가 적용된다는 점을 이용하였다. INT18 saturated bias addition과 ReLU가 모두 단조 함수이므로, 네 값의 max를 먼저 구한 뒤 bias addition과 ReLU를 한 번만 수행해도 결과가 같다. 이에 따라 2×2 tile마다 bias adder 네 개를 두는 대신 하나를 사용하며, requantization도 scalar 결과에만 적용한다.

3.3 Buffer 및 Activation Storage 최적화

3.3.1 Elastic FIFO

서로 다른 latency와 처리율을 갖는 pipeline block 사이에는 valid/ready 기반의 elastic channel을 사용하였다 [6]. Tile FIFO와 M FIFO는 downstream stall 동안 데이터를 보존하며, producer와 consumer가 동시에 준비된 cycle에만 transfer를 수행한다. V buffer는 같은 handshake를 사용하지만, channel tile의 행을 독립적으로 선택하므로 일반적인 순차 FIFO와 구분한다.

각 boundary의 저장 폭과 depth는 data format 및 occupancy 관측에 따라 결정하였다. Tile FIFO에는 UINT8 4×4 tile, V buffer에는 INT11 변환 tile과 zero-row 정보, M FIFO에는 INT18 channel-reduction 결과가 저장된다. V buffer의 행별 선택과 bypass는 3.2.3절에서 설명하였다.

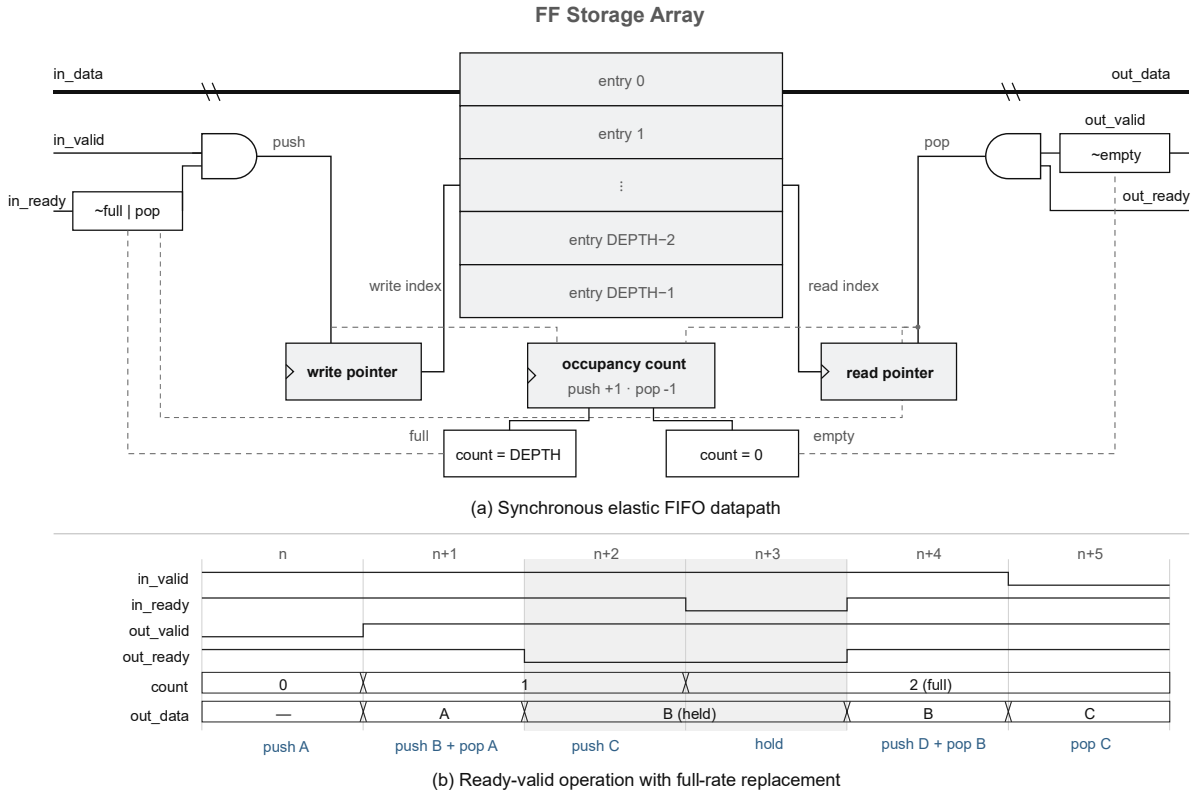


Fig. 11. Elastic FIFO microarchitecture and valid-ready timing.

3.3.2 FIFO Depth 최적화

Buffer depth는 우선 sparsity-aware 확장 이전의 공통 pipeline에서 탐색하였다. Conv1 Tile FIFO의 depth를 1, 2, 4, 8로 바꾸며 maximum occupancy, input backpressure 및 1,000회 RTL 검증 결과를 비교하였다. 이 개발 단계 비교와 최종 통합 설계의 실행 사이클은 구분한다.

Depth 1에서는 core stall을 흡수하지 못해 영상당 36 cycle의 input backpressure가 발생하였다. 대회 input interface는 ready 신호에 따라 pixel 전송을 중단하지 않으므로 pixel alignment가 깨졌으며, accuracy는 9%까지 감소하였다. 반면 Depth 2 이상에서는 maximum occupancy가 2를 넘지 않았고 input stall 없이 97% accuracy를 유지하였다. 이에 따라 필요한 stall 흡수 능력을 유지하는 최소 용량인 depth 2를 채택하였다.

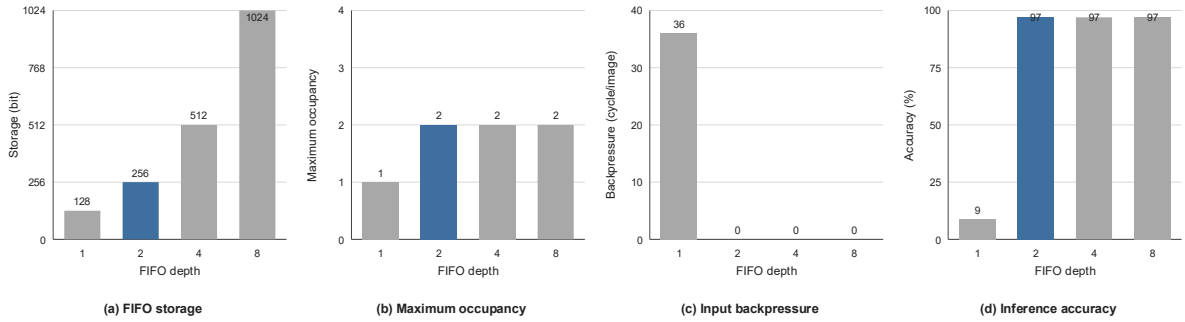


Fig. 12. Conv1 Tile FIFO depth sweep.

Fig. 13에서 Depth 1은 FIFO가 가득 찰 때 image_ready를 deassert한다. Depth 2는 두 번째 entry가 tile을 보존하므로 같은 구간에서도 고정 속도 pixel stream을 수락할 수 있다.

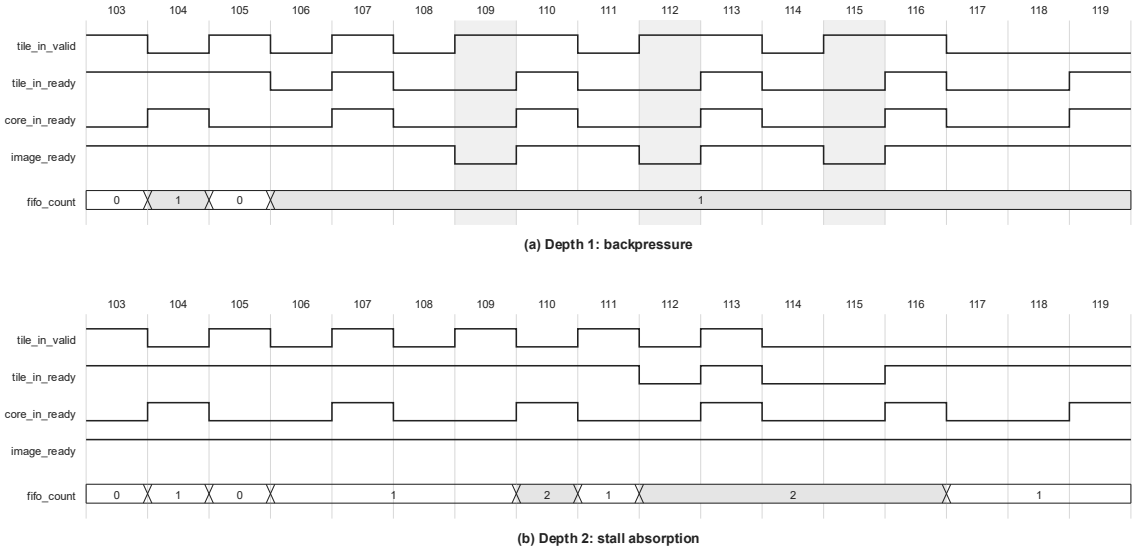


Fig. 13. FIFO occupancy and input backpressure.

V buffer와 M FIFO도 독립적으로 비교하였다. 당시 V buffer depth 1은 input loss를 발생시켜 channel-group depth 2를 유지하였고, M FIFO는 최대 occupancy가 1이므로 depth 1을 선택하였다. Fig. 14와 Table 2에 비교 결과와 최종 구성을 정리하였다.

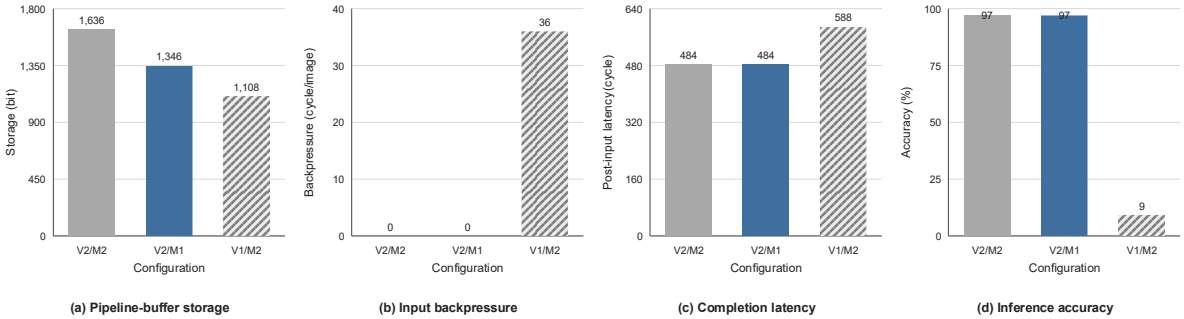


Fig. 14. Pipeline buffer depth comparison in the development configuration.

Buffer	Pipeline boundary	Depth / max occupancy
Tile FIFO	Tile Generator → Input Transform	2 / 2
V buffer	Channel tile groups with row selection	2 / 2
M FIFO	MAC → Output Transform	1 / 1

Table 2. Selected pipeline buffer depths.

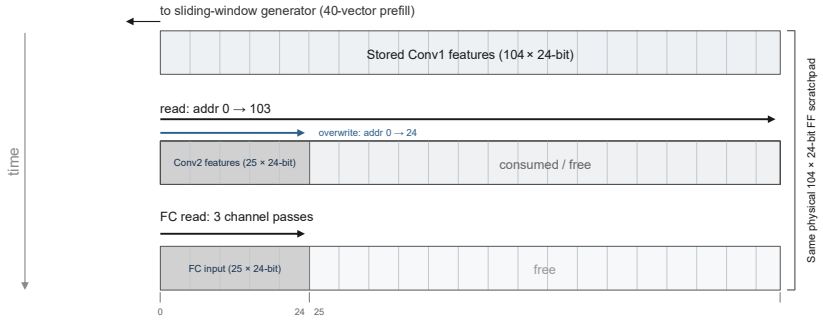
입력 stall을 충분히 흡수하도록 여유 있게 잡은 초기 FIFO 용량을 occupancy 측정 결과에 맞춰 depth 2로 최적화하였다. Depth 8의 여유 용량 구성과 비교하면 payload storage는 1,024 bit에서 256 bit로 75% 감소한다. 동일 조건 합성에서 FF 수는 6.67%, total cell area는 5.97%, vectorless power는 6.62% 감소하였다.

3.3.3 Lifetime-aware Activation Storage

초기 구조에서는 13×13 Conv1 output 전체와 $5 \times 5 \times 3$ Conv2 output을 별도의 buffer에 저장하였다. 그러나 Conv2의 실제 reference 영역과 activation lifetime을 분석하면 두 data를 동시에 유지할 필요가 없다.

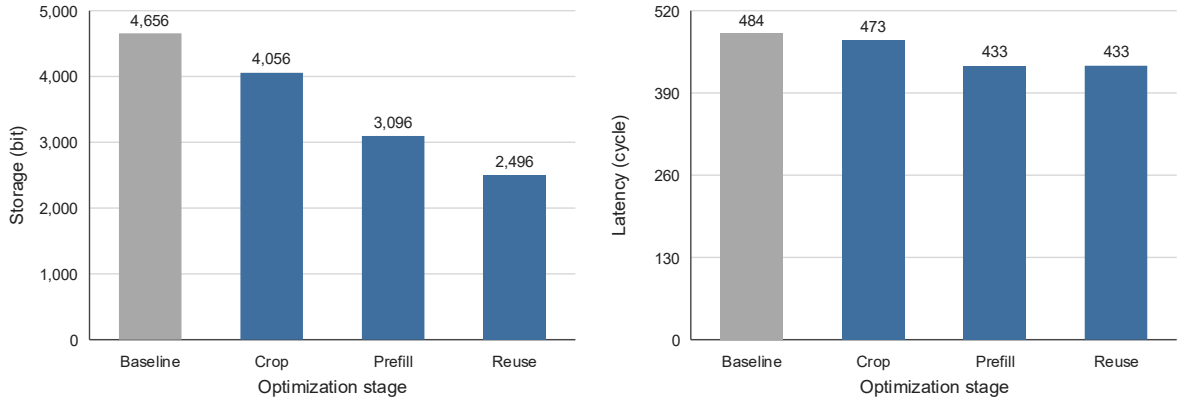
먼저 Conv2에서 참조하지 않는 Conv1 output의 마지막 row와 column을 제거하였다. 이후 첫 번째 4×4 Conv2 window를 구성하는 40개 vector는 Conv1 실행 중 Tile Generator에 직접 prefill하고, 나머지 104개 vector만 Activation Scratchpad에 저장하였다.

Conv2가 scratchpad의 activation을 순차적으로 읽으면 소비가 완료된 address가 발생한다. 본 설계는 이 영역의 address 0–24에 Conv2 output을 덮어쓴다. Conv2 output은 동일 spatial position의 세 channel을 하나의 24-bit word로 packing하며, FC layer에서는 byte lane을 순차적으로 선택하여 channel-major 순서를 복원한다. 이에 따라 별도의 FC feature buffer를 제거하였다.



(b) Lifetime-aware activation storage

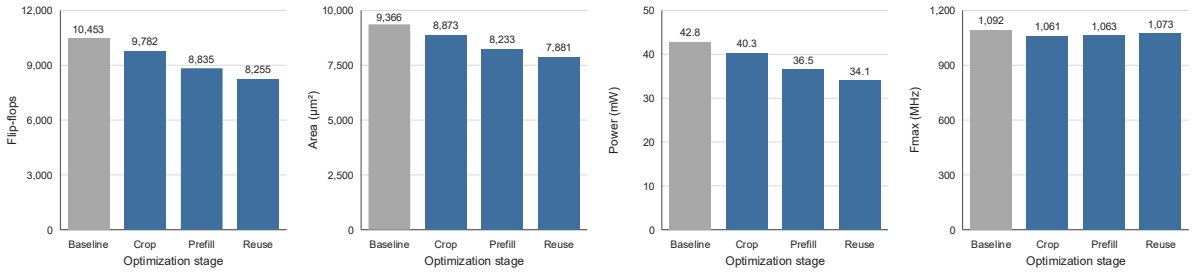
Fig. 15. Consumption and reuse of the same activation storage.



(a) Inter-layer storage **(b) Post-input latency**

Fig. 16. Lifetime-aware activation storage optimization.

Fig. 16의 개발 단계 비교에서 inter-layer storage는 4,656 bit에서 2,496 bit로 46.39% 감소하였다. 마지막 pixel 입력 이후의 처리 사이클도 484에서 433으로 줄었으며, 각 단계에서 RTL mismatch 0과 input backpressure 0을 유지하였다.



(a) Flip-flop count **(b) Cell area** **(c) Vectorless power** **(d) Estimated Fmax**

Fig. 17. Post-synthesis PPA during activation storage optimization.

Consumed-region reuse까지 적용한 당시 회로는 169-entry 기준 대비 FF 수를 21.03%, total cell area를 15.86%, sequential area를 20.08% 줄였다. 같은 vectorless 분석의 전력 감소는 20.33%였다. 이 값은 저장 구조 변경의 국소 비교이며, 최종 ICG 회로의 VCD 기반 전력으로 대체하거나 두 감소율을 합산하지 않는다.

RTL에서는 prefill 동안 ready 신호를 연산기까지 되돌리는 조합 경로를 제거하고, 소비가 끝난 주소만 overwrite하도록 하였다. 저장 배열의 크기뿐 아니라 접근 순서와 제어 경로를 함께 수정하여 기능과 합성 가능성을 확보하였다.

3.4 Standard-cell 합성 및 PPA 최적화

타일 단위 데이터 흐름을 RTL로 구현한 뒤 Yosys와 OpenROAD/OpenSTA로 실제 셀 구성과 critical path를 확인하였다. ICG 셀 매핑과 MAC 제어 조건의 register 위치를 조정하여, 연산 구조뿐 아니라 합성된 회로의 클럭 활동과 timing도 개선하였다.

3.4.1 ICG 셀 매핑 및 gated data register

제공된 ASAP7 RVT library에서 ICGx1_ASAP7_75t_R의 CLK, ENA, SE 및 GCLK 핀과 latch 기반 clock-gating 기능을 확인하였다. 단순한 register enable만으로 클럭 활동이 제거되었다고 간주하지 않고, ASIC 합성 경계에 OPENROAD_CLKGATE를 명시하였다. 합성 실행 시 OPENROAD_CLKGATE define을 설정하고 플랫폼의 cells_clkgate_R.v를 읽어 실제 ICG 셀로 연결하였다. Scan enable SE는 사용하지 않으므로 0에 고정하였다.

```
// RTL boundary: winograd_clock_gate.v
OPENROAD_CLKGATE u_icg (.CK(clk),
    .E(enable), .GCK(gclk));

// ASAP7 platform mapping: cells_clkgate_R.v
ICGx1_ASAP7_75t_R latch (.CLK(CK), .ENA(E),
    .SE(1'b0), .GCLK(GCK));
```

Listing 1. Explicit ASIC clock-gate mapping excerpts.

Fig. 18과 같이 product register는 입력을 수락할 때, accumulator는 누적 결과를 commit할 때, result register는 마지막 입력의 결과를 기록할 때 클럭을 공급한다. Input/Output Transform, post-processing 및 buffer payload에도 같은 원칙을 적용하였다. Valid/ready와 제어 상태는 원래 clk를 사용하고, 데이터 register만 gated clock 영역으로 분리하였다.

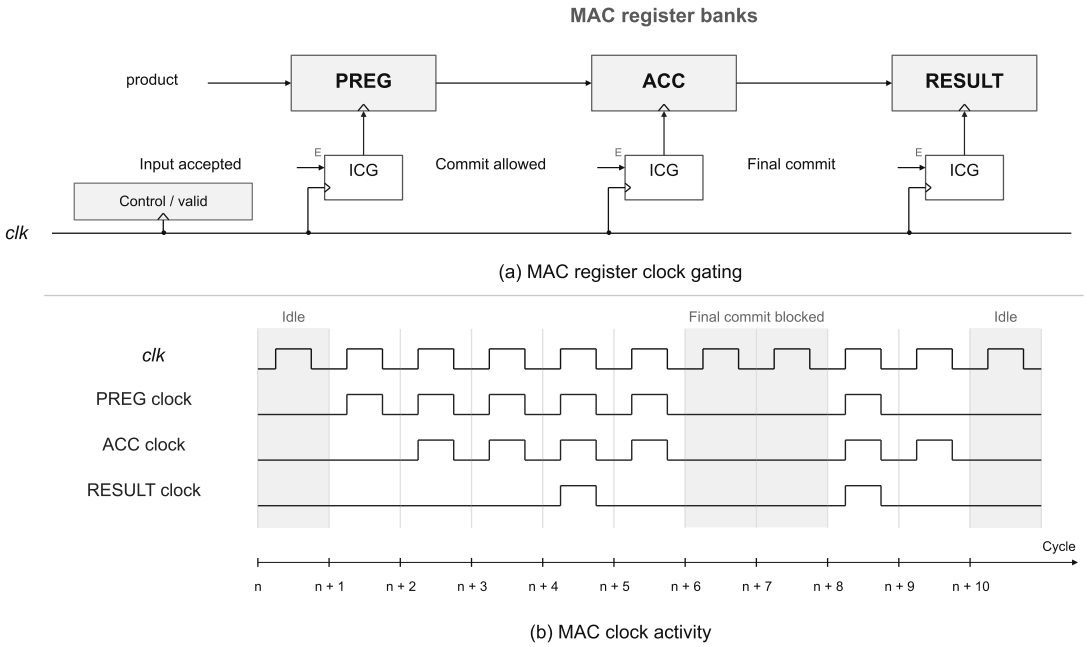


Fig. 18. Clock-enable conditions of the data register banks.

Fig. 19에서 ICG는 클럭이 낮을 때 enable을 latch에 반영하고 높은 구간에서는 유지한다. 이에 따라 enable 변화가 클럭의 high 구간을 직접 절단하지 않는다. RTL simulation에는 latch 기반 기능 모델을, ASIC 합성에는 library cell을 사용하였다.

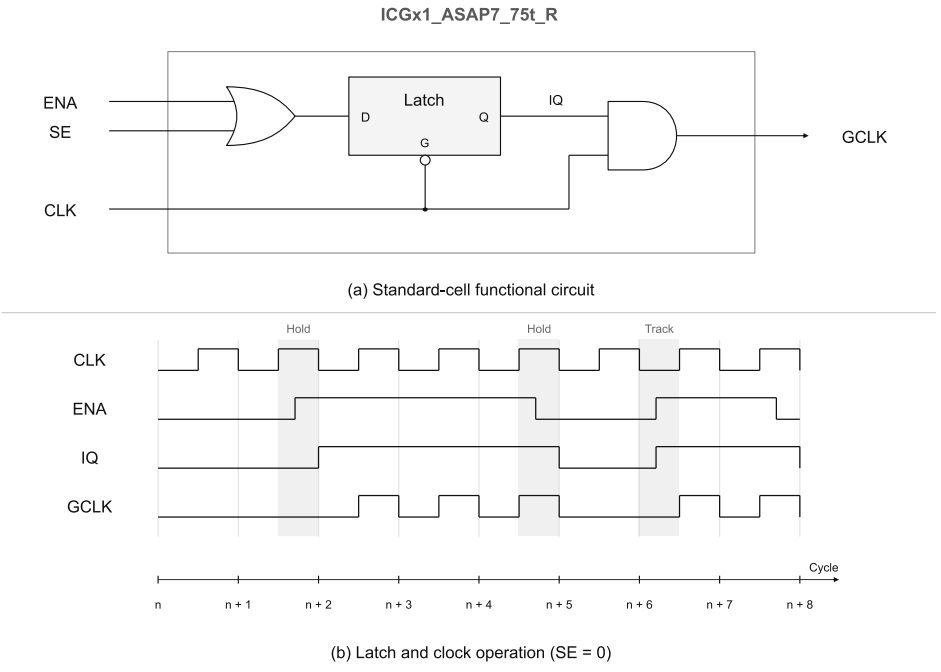


Fig. 19. Latch-based operation of the ASAP7 integrated clock-gating cell.

GCLK 연결 분석에서는 총 7,188개 data-register clock input이 ICG에 연결된 것을 확인하였다. Valid/ready와 제어 상태를 원래 clk에 남겨 둔 채 payload register의 클럭 활동을 억제하였다.

3.4.2 MAC 누산 제어 경로 개선

개발 단계의 timing report에서는 pipe_weight_zero register에서 zero-weight 선택 논리와 포화 누산기를 거치는 경로가 병목이었다. 누산 단계에서 lane 활성 여부와 weight-zero 조건을 다시 조합하지 않도록, 앞단에서 lane_active & ~weight_zero를 계산하여 pipe_accumulate에 저장하였다. 다음 단계는 등록된 조건으로 accumulator 갱신 여부를 직접 선택한다.

```
// Accepted input at the product stage
pipe_accumulate[lane] <= lane_active & ~weight_zero;

// Accumulation stage
assign accumulated_value = pipe_accumulate[lane]
    ? sat_add(accumulator[lane], lane_term)
    : accumulator[lane];
```

Listing 2. Registered accumulation predicate in the MAC datapath.

Table 3의 동일 조건 비교에서 최소 주기 추정치는 946.20 ps에서 847.31 ps로 10.45% 줄었으며, 셀 면적 증가는 0.18%였다. Critical path 시작점이 zero-weight 제어 register에서 accumulator register로 이동한 것도 STA report에서 확인하였다. 파이프라인 단계 수와 handshake 및 수치 연산 순서는 바꾸지 않았고, 당시 RTL 및 gate netlist의 1,000회 회귀에서 불일치 0과 post-input 지연 433 cycle을 유지하였다.

Development configuration	Minimum period (ps)	Cell area (μm²)
Before predicate registration	946.20	7,939.82
Registered pipe_accumulate	847.31	7,953.89
Sign-overflow saturation alternative	966.68	7,744.81

Table 3. Local synthesis comparison for MAC timing optimization.

Sign-overflow 방식의 포화 덧셈은 면적을 줄였지만 최소 주기가 966.68 ps로 늘어 채택하지 않았다. Table 3은 이 개발 단계의 비교이며, sparsity-aware 구조와 ICG까지 통합한 최종 결과는 3.5절에서 제시한다

.

3.5 아키텍처 검증 및 구현 결과

3.5.1 모델 준비 및 기능 검증

본 RTL에서 사용하는 transformed weight의 INT4 양자화를 학습에 반영하여 Winograd-aware W4A8 QAT를 수행하였다 [4, 5]. Layer 사이의 activation은 UINT8, FC weight는 INT4로 구성하였다. 생성한 parameter를 정수 기준 모델과 RTL에 동일하게 적용하고, 같은 연산 순서와 rounding 및 saturation을 사용하여 결과의 일치를 검증하였다.

Table 4는 FP32, PTQ 및 QAT의 정확도 비교이다. Winograd-aware QAT를 적용하여 PTQ 대비 정확도를 회복하고 대회 입력의 목표 정확도를 만족하였다.

Model	Weight	MNIST (10,000)	Competition (1,000)
Spatial 3×3 FP32	FP32	97.03%	—
Winograd W4A8 PTQ	INT4	95.21%	94.00%
Winograd W4A8 QAT	INT4	96.25%	97.00%
Exported integer model	INT4	96.25%	97.00%

Table 4. Accuracy comparison of FP32, PTQ, and QAT models

최종 지정주제 RTL을 외부 pixel 입력부터 Argmax-10까지 정수 기준 모델과 비교하였다. 대회 입력은 고유 이미지 100개를 10회 반복한 1,000회 입력이며, 970/1,000 정답, RTL mismatch 0 및 input backpressure 0을 확인하였다.

합성 후 netlist는 같은 수치 형식을 사용하는 standard-cell 기능 모델로 검증하였다. 전력 분석용 20개 고유 입력에서도 정수 기준 모델과 불일치는 없었다. 최종 기능 검증과 합성 결과는 Table 5에 정리하였다.

Item	Final designated-topic result
Library / corner	ASAP7 RVT FF NLDM / 0.77 V / 0 °C
Analysis stage	Post-synthesis; ideal clock; no routed parasitics
Operating frequency	1 GHz
Minimum clock period / estimated Fmax	860.07 ps / 1,162.70 MHz
Setup timing margin at 1 GHz	+139.93 ps
Total / sequential cell area	7,769.03 / 2,558.29 μm²
Mapped DFF / ICG count	8,314 / 16
Mean / maximum inference cycles	995.83 / 1,009 cycles
Accuracy / RTL mismatch / input backpressure	970/1,000 / 0 / 0

Table 5. Final implementation and verification summary.

Inference cycle은 첫 pixel 수락 상승 에지부터 decision-valid가 발생하는 상승 에지까지 직접 관측하였다. 평균 995.83 cycle은 1 GHz에서 0.99583 μs에 해당한다. 이는 이전 개발 단계에서 사용한 마지막 pixel 입력 이후의 post-input counter와 시작점이 다르므로, 두 값을 같은 latency로 비교하지 않는다.

3.5.2 활동 기반 전력 및 에너지

전력은 동일한 library와 1 GHz 조건에서 gate-level VCD를 annotation하여 분석하였다. 네 조건 모두 동일한 20개 고유 이미지를 사용하고, reset/start를 포함한 전체 simulation 구간을 측정하였다. Bypass와 ICG를 함께 적용한 최종 회로의 total power는 7.868472 mW이며, simulation 구간 20,048.5 ns를 기준으로 이미지당 에너지는 7.887553 nJ이다.

$$E_{image} = P_{avg} \times T_{VCD} / N_{images}$$

VCD annotation의 unannotated pin 수는 0이었다. 다만 zero-delay gate simulation과 ideal clock을 사용하여 delay-dependent glitch, CTS 및 routed wire capacitance는 포함하지 않았다. 따라서 전력은 활동 기반 post-synthesis 추정값으로 제시한다.

3.5.3 Bypass 및 ICG의 효과 분리

Table 6의 네 회로는 Winograd, tile-level operation fusion 및 storage 재사용을 공통으로 적용한다. 동일한 기본 아키텍처에서 sparse bypass와 ICG를 각각 켜고 끄는 2×2 비교를 수행하여 추가 효과를 분리하였다. Bypass에는 row compaction, 전체 zero 작업 우회 및 ordering logic이 포함된다.

Case	Mean cycles	Area (μm²)	Power (mW)	Energy (nJ/image)
Common structure	1,072.00	7,804.64	16.5916	17.8861
Bypass	995.83	7,781.48	16.2739	16.3133
ICG	1,072.00	7,802.57	8.6100	9.2818
Bypass + ICG	995.83	7,769.03	7.8685	7.8876

Table 6. Matched bypass/ICG comparison under identical evaluation conditions.

동일 조건의 재합성 결과에서 sparsity-aware 처리와 ICG를 함께 적용한 최종 회로는 공통 구조 대비 에너지를 55.90% 줄였다. Table 7은 각 기법의 전체 합성 면적 변화를 분리하여 나타낸다.

Table 7. Incremental cell-area changes from matched synthesis comparisons.

적용 조건	면적 변화 (μm²)	적용 전 대비
ICG 추가 (sparsity OFF)	-2.07036	-0.027%
Sparsity 추가 (ICG ON)	-33.54858	-0.430%
Sparsity + ICG 동시 적용	-35.61894	-0.456%

동일 조건 재합성에서 ICG와 sparsity-aware 처리의 추가에 따른 순 면적 증가는 관측되지 않았으며, 면적 변화는 0.5% 미만이었다. 이는 추가 회로와 합성 매핑 변화가 함께 반영된 결과이다. ICG 셀 16개의 면적은 4.19904 μm²로 최종 면적의 0.054%에 해당하며, 작은 면적 비용으로 클럭 활동을 억제한다.

Fig. 20과 같이 ICG를 공통 적용한 회로에서도 bypass를 추가하면 이미지당 에너지가 15.02% 감소하였다. 이는 클럭 활동 억제에 더해 연산 압축과 zero 작업의 우회가 추가 이득을 제공함을 보여준다.

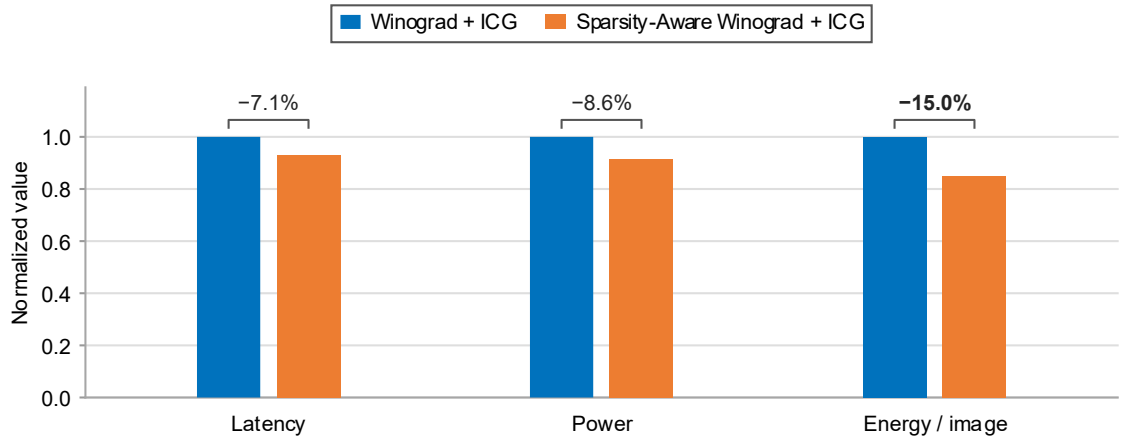


Fig. 20. Additional bypass benefit relative to the common ICG-enabled configuration.

3.5.4 연산 슬롯 감소와 실행 사이클의 관계

Fig. 21에서 Conv1의 활성 MAC lane 슬롯은 57.82% 줄었지만 RUN 구간은 752.00에서 751.44 cycle로 거의 같았다. 정상 MAC 입력 432.00회가 MAC 수락 211.47회와 zero 작업 우회 수락 220.53회로 나뉘어, 수락 작업량의 합계가 여전히 432회였기 때문이다.

Conv2에서는 여러 input channel의 행을 압축할 수 있어 MAC 입력이 225.00에서 121.14회로 줄었다. 전체 zero 우회 6.90회를 포함해도 수락 작업량은 128.04회이며, RUN 구간은 238.00에서 162.39 cycle로 31.77% 감소하였다. Conv1의 0.56 cycle과 Conv2의 75.61 cycle 감소를 더하면 전체 평균 지연 감소인 76.17 cycle과 일치한다.

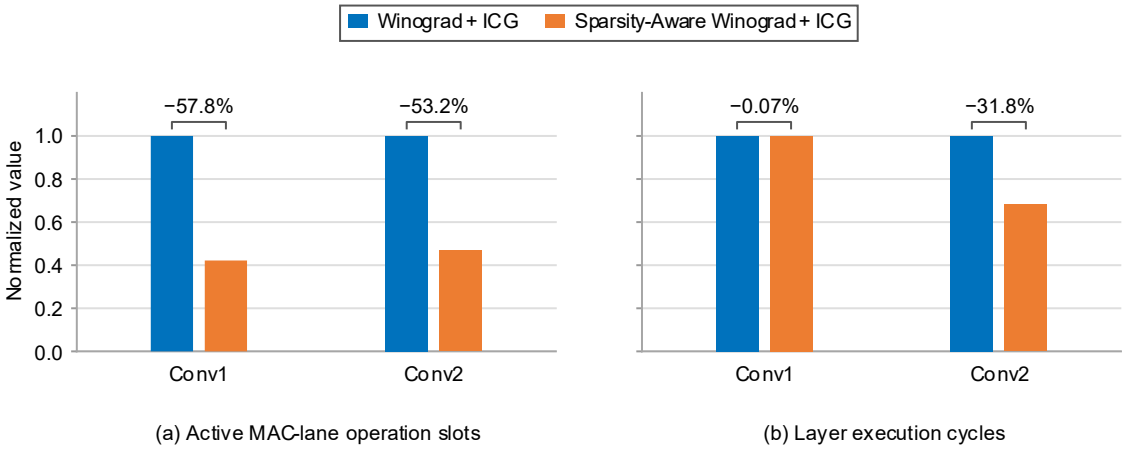


Fig. 21. Measured MAC lane-slot reduction and layer RUN-cycle reduction.

Conv1은 외부 인터페이스에서 cycle당 1 pixel을 받아 tile을 생성한다. 연산을 생략해도 다음 입력의 도착을 기다려야 하므로, MAC 활동 감소가 실행 사이클 절감으로 거의 이어지지 않았다. 반면 Conv2는 scratchpad에 저장된 activation을 내부 경로로 공급받고 세 입력 channel의 유효 행을 압축하여 처리하므로 RUN 사이클이 31.77% 감소하였다. 이 결과를 바탕으로 자유주제에서는 SRAM과 코어 간 공급 구조를 확장하고, 공급 속도에 따른 sparsity-aware 실행의 효과를 추가로 다룬다.

3.5.5 대회 baseline 및 가상 dense 기준 지표

Table 8은 대회 baseline과 제안 아키텍처의 통합 구현 결과이며, kernel 재구성과 하드웨어 구조 변경이 모두 포함된다. Baseline의 전력 활동 조건은 재현하지 않았으므로, bypass·ICG의 개별 추가 효과는 Table 6의 동일 조건 실험으로 평가하였다.

대회 비교 관례인 가상 dense-MAC 지표는 식 (8), (9)로 계산하였다. 기준 MAC 수는 baseline 348, 본 모델 183이며 물리적인 16개 MAC lane과 구분한다. 이 값은 실제 inference 처리량이나 sustained throughput을 뜻하지 않는다.

$$TOPS = (2 \times N_MAC \times f) / 10^{12} \tag{8}$$

$$TOPS/W = TOPS / P \tag{9}$$

Metric	Competition baseline (reported)	Final proposed
Accuracy on competition input	97.0%	97.0%
Operating frequency	476 MHz	1 GHz
Cycles per image	1,335	995.83 (mean)
Inference latency	2.80 μs	0.99583 μs
Cell area	0.024848 mm²	0.007769 mm²
Power	335 mW	7.868 mW
Energy with stated VCD window	Not reproduced	7.888 nJ/image
Virtual dense-reference index	0.331 TOPS	0.366 TOPS
Virtual dense-reference efficiency	0.989 TOPS/W	46.51 TOPS/W

Table 8. Reported baseline and final proposed design with explicit metric boundaries.

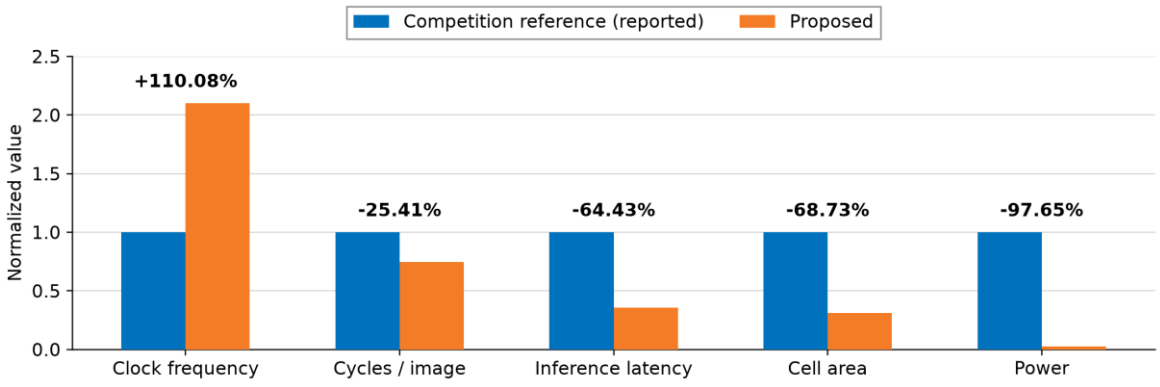


Fig. 22. Normalized implementation metrics and percentage changes relative to the reported competition reference.

Fig. 22는 Table 8의 대회 보고값을 1로 정규화한 비교이다. 주파수는 110.08% 증가하고, 이미지당 사이클은 25.41%, 지연시간은 64.43%, 면적은 68.73% 감소하였다. 전력 보고값의 차이는 97.65%이며 두 회로의 활동 조건은 다르다. 두 모델의 대회 입력 정확도는 모두 97.0%이다.

4. 결론

본 과제는 activation의 zero 비율에 의존하지 않는 Winograd 연산 감소를 기반으로, 타일 생성부터 channel reduction과 후속 처리까지 이어지는 가속기 아키텍처를 구현하였다. 공유 16-lane MAC, tile-local fusion 및 activation storage 재사용으로 연산 자원과 데이터 이동을 함께 설계하고, sparsity-aware 처리와 ICG를 추가하였다.

최종 회로는 1 GHz에서 평균 995.83 cycles/image, 7,769.03 μm²의 cell area 및 7.868 mW의 활동 기반 추정 전력을 보였다. 동일 조건의 기법 분리 비교에서 sparsity-aware 처리와 ICG는 순 면적 증가 없이 평균 사이클을 7.11%, 전력을 52.58%, 이미지당 에너지를 55.90% 줄였다. ICG 적용 회로에 sparsity-aware 처리를 추가한 경우에도 에너지가 15.02% 감소하였다. 이 결과는 매우 작은 면적 비용으로 불필요한 연산과 클럭 활동을 함께 줄일 수 있음을 보여준다.

5. 참고문헌

[1] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[2] A. Lavin and S. Gray, "Fast Algorithms for Convolutional Neural Networks," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4013–4021, 2016.

[3] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-Layer CNN Accelerators," *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 1–12, 2016.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2704–2713, 2018.

[5] J. Fernandez-Marques, P. N. Whatmough, A. Mundy, and M. Mattina, "Searching for Winograd-aware Quantized Networks," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 14–29, 2020.

[6] J. Cortadella, M. Kishinevsky, and B. Grundmann, "Synthesis of Synchronous Elastic Architectures," *Proceedings of the 43rd Design Automation Conference*, pp. 657–662, 2006.

[7] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *International Conference on Learning Representations*, 2015.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.

[9] A. Parashar et al., "SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks," *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pp. 27–40, 2017.